

CG2271 Real Time Operating Systems

2025/26 Semester 2

Final Report

I. Problem statement

Design and implement a cloud-connected weather monitoring system using ESP32 and MCXC444 that measures temperature, humidity, and water level while detecting potential disaster hazards such as earthquakes, tsunamis, and fire, and transmitting the data to an online dashboard.

Feature List

Environmental monitoring: temperature, humidity, water level measurement

Disaster detection: fire hazard, seismic activity and tsunami detection

Local alerts: audio alarm via buzzer with external RGB visual indication

II. Sensors and Actuators used

Sensor	Description of use	Libraries used (if any)
DHT11	Temperature and humidity sensor is used to monitor ambient environmental conditions, of which data is sent to MCXC444 via UART	DHT11.h by Dhruba Saha <u>GitHub - dhrubasaha08/DHT11: DHT11 Arduino Library: A simple and efficient library for reading temperature and humidity data from the DHT11 sensor without dependencies.</u>
Flame Sensor	Infrared flame detection is used to detect fire hazards via GPIO interrupt	
Shock Sensor	Shock sensor spring mechanism is used to detect seismic activity or vibration via GPIO interrupt	

Ultrasonic Sensor	Distance measurement is used to monitor water level, which then sent to ESP32	
-------------------	---	--

Actuators	Description of use	Libraries used (if any)
Buzzer	Audio alert is used to provide distinctive alarm patterns for fire, shock, water	
External RGB LED	Visual indicator with priority-based control: red for fire, green for shock, blue for water	
Internal RGB LED	Temperature status indicator: red when temperature > 24°C, blue when temperature 22.5–24°C ,green when temperature < 22.5°C	MCUXpresso Software Development Kit (SDK)

III. Hardware Architecture

The system uses a dual-microcontroller design with an ESP32 and an MCXC444. The MCXC444 handles real-time sensor inputs (shock, flame, and water level) using interrupts and polling, and sends event/data to the ESP32 via UART. The ESP32 acts as the main controller, reading temperature and humidity, controlling the buzzer and LEDs, and uploading data to Firebase over WiFi. This separation ensures that time-critical sensing is not affected by network delays, improving system reliability and responsiveness. The following figures (Fig.1, Fig. 2, Fig. 3) show a detailed pin connection and circuit diagram for our project setup.

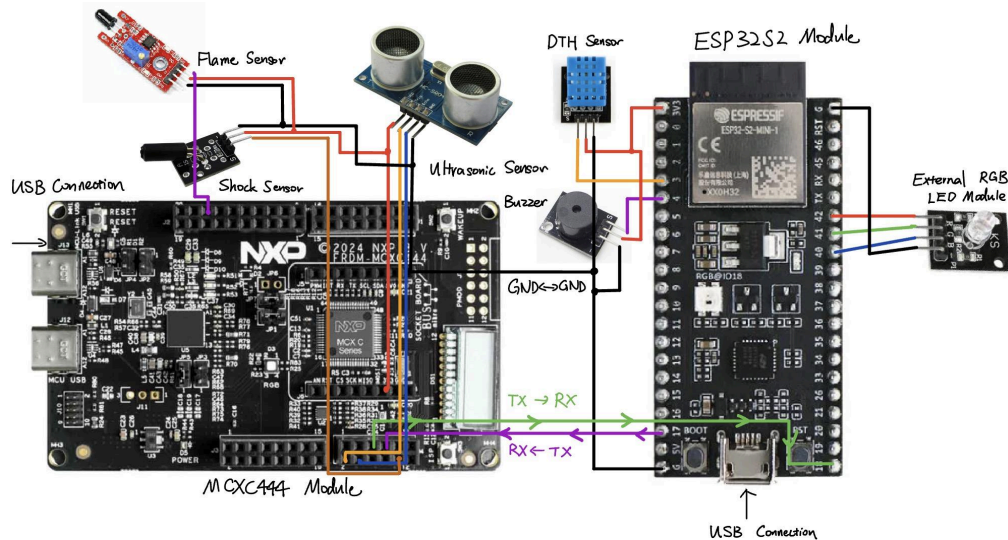


Figure 1. Circuit Diagram for the Project

MCXC444 Peripherals	Port/Pin	Direction	Description
UART TX → ESP32	PTE22	Output	Sends data to ESP32 via UART
UART RX ← ESP32	PTE23	Input	Receives data from ESP32 via UART
Internal Red LED	PTE31	Output	Temperature indicator
Internal Green LED	PTD5	Output	Temperature indicator
Internal Blue LED	PTE29	Output	Temperature indicator
Shock Sensor	PTC2	Input	Detects vibration
Ultrasonic Trigger	PTB0	Output	Sends pulse
Ultrasonic Echo	PTB1	Input	Measures distance
Flame Sensor	PTD4	Input	Detects fire

Figure 2. Pin Table Layout for MCXC444

ESP32 Peripherals	GPIO Pin	Direction	Description
-------------------	----------	-----------	-------------

DHT11 Sensor	GPIO 3	Input	Reads temperature & humidity
Buzzer	GPIO 4	Output	Plays different alarm sounds
UART TX → MCXC444	GPIO 17	Output	Sends data to MCXC444
UART RX ← MCXC444	GPIO 18	Input	Receives data from MCXC444
External LED (Red)	GPIO 42	Output	Disaster indicator (Fire)
External LED (Green)	GPIO 41	Output	Disaster indicator (Shock)
External LED (Blue)	GPIO 40	Output	Disaster indicator (Water)

Figure 3. Pin Table Layout for ESP32

The sensors and actuators are programmed using FreeRTOS tasks. On the ESP32, the DHT11 is polled periodically to obtain temperature and humidity readings, which are sent to the MCXC444 through UART. The ESP32 also receives disaster messages such as SHOCK, FIRE, and DIST from the MCXC444, then activates the buzzer and external RGB LED accordingly, while also uploading the latest data to Firebase. On the MCXC444, the shock sensor and flame sensor use interrupts for fast event detection, while the ultrasonic sensor is polled regularly to monitor water level. The MCXC444 also receives temperature data from the ESP32 and uses it to control its onboard RGB LED based on predefined temperature ranges.

IV. Software Architecture

Task Structure

On MCXC444, the software is structured using FreeRTOS Template with both time-slicing and pre-emption enabled. The project follows RTOS multi-tasking design where each functional module is encapsulated into an independent task. There are total of 5 main tasks implemented: uartRecvTask, ledTask, shockTask, fireTask, waterLevelTask (Fig.4)

```
xTaskCreate(uartRecvTask, "uartRecvTask", configMINIMAL_STACK_SIZE + 150, NULL, 3, NULL);
xTaskCreate(ledTask, "ledTask", configMINIMAL_STACK_SIZE + 100, NULL, 2, NULL);
xTaskCreate(shockTask, "shockTask", configMINIMAL_STACK_SIZE + 100, NULL, 3, NULL);
xTaskCreate(waterLevelTask, "waterLevelTask", configMINIMAL_STACK_SIZE + 150, NULL, 2, NULL);
xTaskCreate(fireTask, "fireTask", configMINIMAL_STACK_SIZE + 100, NULL, 3, NULL);
```

Figure 4. Illustration of Tasks created in MCXC444

uartRecvTask: The task is responsible for receiving and processing temperature and humidity data from ESP32 via UART. The task is blocked until new data at uartRxQueue is available. This message string is copied into &msg, which then be parsed and extracted to get temperature and humidity

data. These processed data will be stored in tempQueue and printed out to the user's terminal.

ledTask: The task is responsible for controlling RGB LED output based on sensor readings. Once the new received message is available at tempQueue, it will be copied into &sensor and later parsed into `setLedFromTemperature()` for LED pins toggling.

shockTask: The task is blocked until the “shock” event where the ISR gives the shockSem semaphore. The task will then print "Shock detected!\r\n" to user and send "SHOCK\r\n" message to ESP32 through UART.

fireTask: The task is blocked until the “fire” event whenever the ISR gives the fireSem semaphore. It then sends a "FIRE\r\n" message to ESP32 via UART.

waterLevelTask: The task measures the distance to the water surface. This function periodically calls `measureDistanceCm(void)` to trigger the ultrasonic sensor and return distance in centimetres. The distance data is then printed to the user and sent as a message under format "DIST:%d.%d\r\n" to ESP32 via UART. The task is then suspended for 1000ms until next run, setting its reporting and measuring rate to be once per second.

On the ESP32, the software is also structured using FreeRTOS, where the system is broken down into multiple concurrent tasks so that sensing, communication, actuation, and cloud uploading can proceed independently rather than blocking each other. The ESP32 contains six main tasks: dhtTask, uartSendTask, uartRecvTask, buzzerTask, ledTask, and firebaseTask. These tasks are explicitly created using `xTaskCreate(...)`, showing a multitasking RTOS design (Fig. 5)

```
420 xTaskCreate(dhtTask, "dhtTask", 2048, NULL, 2, NULL);
421 xTaskCreate(uartSendTask, "uartSendTask", 2048, NULL, 2, NULL);
422 xTaskCreate(uartRecvTask, "uartRecvTask", 2048, NULL, 3, NULL);
423 xTaskCreate(buzzerTask, "buzzerTask", 2048, NULL, 2, NULL);
424 xTaskCreate(ledTask, "ledTask", 2048, NULL, 3, NULL);
425 // Store handle so uartRecvTask can notify it directly on critical events
426 xTaskCreate(firebaseTask, "firebaseTask", 8192, NULL, 1, &firebaseTaskHandle);
```

Figure 5. Illustration of Tasks created in ESP32

The **dhtTask** periodically reads temperature and humidity from the DHT11 sensor every 2 seconds using DHT.h library. It then stores the values in shared global variables latestTemp and latestHum, and also places the reading into dhtQueue using `xQueueOverwrite`. This means the task acts as the producer of temperature and humidity data. The `uartSendTask` acts as the consumer of this queue. It blocks on `xQueueReceive` until a new sensor reading is available, formats the reading into a UART message like TEMP:22.5,HUM:69.8, and transmits it to the MCXC444. This separates sensing from communication, so the DHT sampling logic does not wait for UART transmission to finish.

The **uartRecvTask** is responsible for receiving messages from the MCXC444 to the ESP32. It continuously checks the UART buffer, reconstructs complete newline-terminated messages, and interprets three incoming events: SHOCK, FIRE, and DIST:x. When a SHOCK message is received, it sets `shockEventLatched` and `shockAlarmPending` flags; when a FIRE message is received, it sets `fireEventLatched` and `fireAlarmPending` flags. For distance, it determines if the water is above threshold, and computes a water-level percentage for Firebase upload.

The actuation layer is handled by two separate tasks: **buzzerTask** and **ledTask**. The `buzzerTask` checks the system state and decides which alarm to play. It gives priority to fire, then shock, then water. If a fire event is detected, it clears the pending flag and plays the fire siren. If not, it checks for shock. Else, if the water threshold condition is active, it plays the water alarm. The `ledTask` works in parallel and activates external RGB LED according to the same disaster state. It periodically calls `setDisasterLed(fireAlarmPending, shockAlarmPending, waterBelowThreshold)` so that the LED provides continuous visual feedback while the buzzer provides audio feedback.

The **firebaseTask** implements the online function of the ESP32. Its purpose is to upload the latest system data to Firestore. This task is intentionally isolated from the others so that cloud communication never stalls sensor handling or UART reception. The task wakes either every 5 seconds for a periodic heartbeat upload or immediately when a critical event notification like SHOCK or FIRE is sent from `uartRecvTask`. This is an efficient event-driven design because emergency events do not need to wait for the next fixed upload interval.

Coordination and Synchronisation

```
QueueHandle_t uartRxQueue;  
QueueHandle_t tempQueue;  
  
SemaphoreHandle_t shockSem;  
SemaphoreHandle_t fireSem;  
SemaphoreHandle_t uartMutex;
```

Figure 6. Queues, Semaphores Mutex used in MCXC444

For MCXC444, the coordination between tasks and interrupts is achieved through usage of queues, semaphores and a mutex as shown in Figure 6.

uartRxQueue is used to transfer incoming UART data from UART ISR to `uartRecvTask`. When data is received, the UART ISR pushes the received message to `uartRxQueue`. The `uartRcvTask` is blocked on `xQueueReceive()` until new data is available. This approach ensures minimal processing inside the ISR and prevention of data loss during high-frequency UART communication.

tempQueue is created with length 1 and is used to transfer processed sensor data from `uartRecvTask` to `ledTask`. Since only the most recent temperature value is

required for LED control, the system uses `xQueueOverwrite()` to replace older data in the queue. This ensures that `ledTask` always processes the latest available data while no outdated sensor readings accumulate.

Binary semaphores (**shockSem**, **fireSem**) are used for interrupt-driven synchronization between GPIO interrupts and their corresponding tasks (`shockTask`, `fireTask`). When an event occurs, the GPIO ISR calls `xSemaphoreGiveFromISR()` to signal the event. The corresponding task, which was blocked on `xSemaphoreTake(..., portMAX_DELAY)`, is immediately unblocked. This ensures efficient CPU usage when no event occurs and immediate handling once detection.

The **uartMutex** is used to control access to the shared UART transmission resource between multiple tasks (`shockTask`, `fireTask`, `waterLevelTask`). The task must acquire mutex before sending a message and release it once complete transmission. This ensures synchronization by effectively serialising UART access across all tasks.

```
QueueHandle_t dhtQueue;
SemaphoreHandle_t uartMutex;
TaskHandle_t firebaseTaskHandle = NULL;

typedef struct {
    float temperature;
    float humidity;
} TSensorData;
```

Figure 7. Queue and Mutexes used in ESP32

For ESP32, coordination between tasks is achieved through the use of a queue, along with FreeRTOS scheduling mechanisms as shown in Figure 7. The `dhtQueue` is created with a length of 1 and is used to transfer sensor data from the `dhtTask` to the `uartSendTask`. Since only the most recent sensor reading is required, the system uses `xQueueOverwrite` to replace older data, ensuring that the consumer always processes the latest available value. This design prevents unnecessary queue buildup and maintains efficient, real-time data flow between tasks. The sensor data is first encapsulated in a structured format, `TSensorData`, which contains both temperature and humidity values. This structured representation serves as the internal packet format before being converted into a string for transmission.

Resource Allocation

For MCXC444, the mutex `uartMutex` also has an effect on resource protection for the UART transmit. The `uartMutex` ensures only one task accesses the UART buffer at a time, preventing message corruption or interleaved transmission from multiple tasks attempting to use UART at same time.

Regarding task priorities, MCXC444 assigns higher priority to time-sensitive and interrupt-driven tasks, while lower priority is given to polling-based and non-critical tasks. Specifically, `uartRecvTask`, `shockTask`, and `fireTask` are assigned the highest

priority level of 3, as they are responsible for real-time data handling and immediate response to critical events. In contrast, polling-based tasks such as `waterLevelTask`, as well as non-time-sensitive tasks like `ledTask`, are assigned a lower priority of 2, ensuring no interference against critical events and incoming data response.

For ESP32, the mutex `uartMutex` is used for resource protection on the UART transmit. Before `uartSendTask` writes to `mySerial`, it calls `xSemaphoreTake(uartMutex, portMAX_DELAY)`, and after transmission it releases the mutex with `xSemaphoreGive(uartMutex)`. This prevents multiple tasks from writing to the same UART peripheral at the same time and corrupting outgoing serial data. The mutex is a RTOS design to protect the shared communication resource and allows future expansion safely.

In terms of task priorities, the ESP32 gives higher priority to time-sensitive local tasks than to the network task. `uartRecvTask` and `ledTask` are created with priority 3 (highest), `dhtTask`, `uartSendTask`, and `buzzerTask` are created with priority 2, and `firebaseTask` runs at priority 1 (lowest). This ensures that emergency messages and local alerts should happen instantly, while cloud uploading is less urgent and should not interfere with real-time responsiveness.

V. Communication Protocol

UART Communication

For the communication protocol between the ESP32 and MCXC444, the system uses UART at 9600 baud and exchanges simple text-based messages. It specifies the frame structure with 8 data bits, no parity bit, and 1 stop bit, which is a standard and widely used UART configuration. Outgoing messages from the ESP32 to the MCXC444 carry DHT readings in the format `TEMP:<value>,HUM:<value>\n`, while incoming messages from the MCXC444 include `SHOCK`, `FIRE`, and `DIST:<value>`. This protocol is lightweight and human-readable on the serial monitor, sufficient for exchanging sensor and event information between the two boards.

Packet Description

The communication between the ESP32 and MCXC444 is based on a structured data representation centred around the `TSensorData` format. On the ESP32 side, temperature and humidity readings obtained from the DHT11 sensor are first encapsulated into a `TSensorData` structure, which serves as the internal data format for processing and task communication. This structured data is then converted into a string-based packet before being transmitted via UART to the MCXC444. Upon reception, the MCXC444 parses the incoming data and extracts the relevant sensor values for further processing or decision-making. In the reverse direction, the MCXC444 sends event-based messages such as sensor triggers or measurement

updates back to the ESP32, which are then interpreted by the UART receive task. This bidirectional exchange ensures consistent and organised communication, with `TSensorData` acting as the primary internal representation of sensor information on the ESP32, while both systems rely on a simple and compatible packet structure for data transmission.

VI. Online function

Two online functions were implemented in this project: writing sensor data to a cloud database via the ESP-Firebase library, and querying the OpenAI API to perform inference on that data.

Firebase Integration

On startup, the ESP connects to a configured Wi-Fi source. Once connected, it continuously writes sensor readings - temperature, humidity, water level, shock, and fire - to our Firebase Database every 5 seconds. As network writes introduce variable latency, this task was assigned the lowest priority to ensure it remains non-blocking.

Data is written to two separate collections. The first stores only the latest sensor values, supporting a real-time monitoring dashboard. The second maintains a timestamped history of all readings, enabling trend analysis and providing the data foundation for any predictive modelling or forecasting work done at home base. Both collections are accessible through a web interface built alongside the system.

OpenAI API Integration

The second online function leverages the OpenAI API to make sense of incoming sensor data. As a proof of concept, the web interface allows users to trigger an inference request, which filters the last hour of readings by timestamp and passes them to the API. The model then generates a natural language summary of the data, flags any anomalies detected across the sensors, and provides assessments that can inform decisions about whether further action or escalation is warranted. This helps to close the loop between raw sensor data and actionable insight.

VII. Special features

Disaster-Specific Buzzer Alarm Patterns

Each disaster type produces a distinct audible pattern for immediate identification without visual confirmation: fire uses high-frequency alternating tones (3kHz/2kHz), shock uses mid-frequency alternating tones (1.5kHz/2.5kHz), and water uses low-frequency sustained tones (800Hz). A priority cascade (fire > shock > water) ensures only the highest-severity alarm plays when multiple events overlap

Using ultrasonic sensor to detect water level

Instead of using a water level sensor to detect floods, we used an ultrasonic sensor to detect the distance from the sensor to the water level itself and we set the maximum distance to 30 cm to prevent anomalies. The ultrasonic sensor will then send the output to the ESP32 and the ESP32 will calculate the percentage of danger based on the max value of 30 cm and will activate the alarm and notify the firebase when it reaches a value above 83%.

Dual-Flag State Management

Each critical event (fire, shock) uses two separate flags: an alarm flag consumed immediately by the buzzer/LED tasks, and a latched flag that persists until Firebase confirms a successful upload. This ensures Firebase always captures the event as true, even if the buzzer has already consumed and cleared its own flag before the upload occurs.

VIII. Challenges faced & Solutions

1. LED Persistence After Firebase Update

Problem: External RGB LED remained on even after the disaster event had passed and Firebase had already updated to reflect the resolved state.

Root Cause: Originally, a single flag (firePending, shockPending) was shared between the buzzer, LED, and Firebase tasks. When dual flags were introduced to decouple local actuation from cloud updates, the LED task was incorrectly set to read the EventLatched flags (fireEventLatched, shockEventLatched) instead of the AlarmPending flags. Since the latched flags only clear after a successful Firebase upload (which can take 1-3 seconds due to network latency), the LED would stay on long after the buzzer had finished and the event had passed.

Solution: Changed ledTask to read fireAlarmPending and shockAlarmPending instead of the latched flags. These flags are consumed quickly by the buzzer task, so the LED now turns off shortly after the alarm finishes playing, matching the actual event duration.

2. Multi-Sensor State Management

Problem: Behavior undefined when multiple sensors triggered simultaneously (fire + water).

Root Cause: Tasks racing to update global state without priority scheme.

Solution: Established cascade priority in code: both LED and buzzer tasks check `fireAlarmPending` first, then `shockAlarmPending`, then `waterBelowThreshold`. All flags are sent to Firebase simultaneously for complete event logging.